

From sequences to chatbots

How translation models turned into machines you can talk to



Seq2seq · Attention · Transformers · Chat

One task, many names

A sequence goes in. A different sequence comes out.

1

Translation

English sentence in.
French sentence out.

Different length, different word
order.

2

Summarisation

A long article in.
Three sentences out.

3

Conversation

Everything said so far in.
The next reply out.

This is the one we are building
towards.

Why this is harder than it looks

Our running example for the rest of the deck

INPUT (English)

the

black

cat

sleeps

OUTPUT (French)

le

chat

noir

dort

Same meaning, different order. The model has to learn the mapping — it cannot copy word by word.

The output is not word-for-word

"black" is the 2nd English word but "noir" is the 3rd French word. The model has to reorder.

Lengths do not have to match

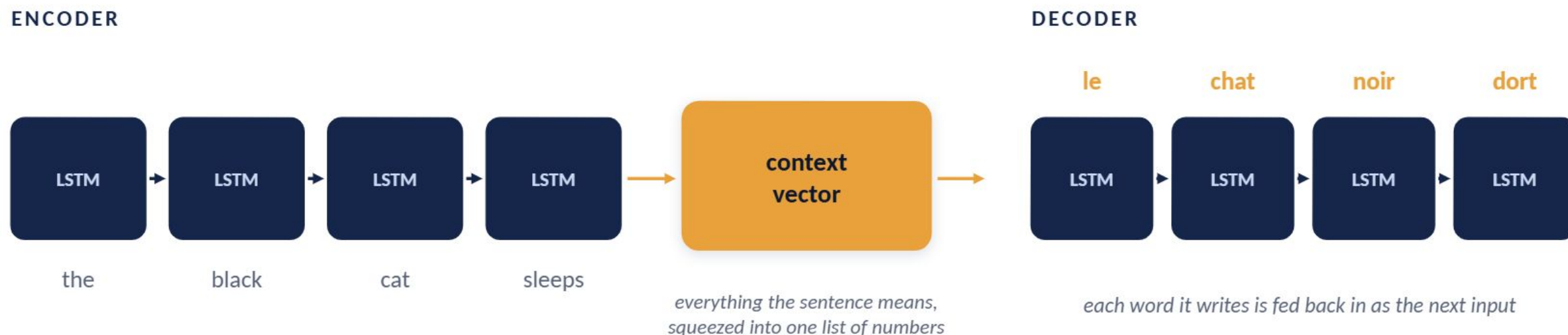
Four words here, but "I am hungry" becomes just "j'ai faim".

Meaning is spread across the sentence

You cannot pick the French for "noir" without knowing the noun it describes is masculine.

Sequence-to-sequence: two networks

One reads. One writes. A single vector passes between them.



The boxes are the same network used again and again — not four different networks.

What actually happens, in three moves

1

Read

The encoder takes one word at a time and updates a running summary. After "the black cat sleeps" it has seen all four words.

2

Compress

The summary after the last word becomes the context vector — a fixed-size list of numbers, say 512 of them, whatever the sentence was.

3

Write

The decoder starts from that vector and produces one French word at a time, feeding each word it writes back in, until it emits a stop token.

PROBLEM

One vector has to carry everything

And that stops working the moment sentences get long.

The bottleneck

Same sized vector, whether the sentence is four words or forty.

"The black cat that had been sleeping on the warm stone wall behind my grandmother's house since early morning finally woke up."



512
numbers

By the time the encoder reaches the last word, the first words have been overwritten many times over. **Translation quality falls off a cliff as sentences get longer.**

The human test

Could you translate a paragraph after reading it once, with the page taken away?

You would want to glance back. So does the model.

Attention: let the decoder look back.

Instead of one summary of the whole sentence, keep the encoder's summary after every word — and let the decoder choose which ones to use for each word it writes.

What the model looks at, word by word

Darker means "this input word mattered more for this output word".

French output	English input			
	the	black	cat	sleeps
le	0.70	0.10	0.15	0.05
chat	0.10	0.10	0.70	0.10
noir	0.05	0.75	0.15	0.05
dort	0.05	0.05	0.10	0.80

Read the third row.

When the model writes "noir", 75% of its attention is on "black" — even though "black" is in a different position in the sentence.

Nobody programmed this alignment. It is learned, as a side effect of training on translation pairs.

How the model picks those numbers

Repeated fresh for every single word it writes.

1

Score

Compare "what I am trying to write right now" against every input word. High score = relevant. It is a similarity measurement, nothing more exotic.

2

Normalise

Push the scores through a softmax so they are all positive and add up to 1. Now they are proportions — the numbers in the grid.

3

Blend

Take a weighted average of the encoder's word summaries using those proportions. That blend is the context for this one output word.

What changed

Without attention

- One context vector for the whole sentence
- Long sentences degrade badly
- No way to see what the model used
- Fixed memory, growing input

With attention

- A fresh context for every output word
- Length stops being the limiting factor
- The weights are inspectable — you can plot them
- Every input word is one step away

NEXT STEP

If attention is doing the work, why keep the RNN?

Attention is all you need, 2017.

Recurrence was the last bottleneck

Not accuracy this time — speed.

RNN — sequential

one step at a time

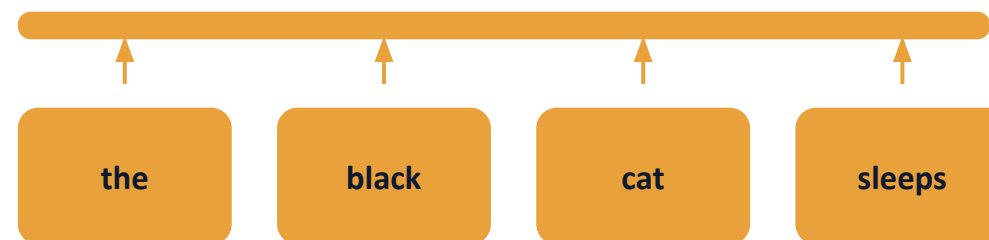


Word 4 cannot start until word 3 is finished.

Training time grows with sentence length, and no amount of hardware fixes it.

Transformer — all at once

every word sees every word



Every word looks at every other word in one step.

All four are computed simultaneously, so the work spreads across as many GPUs as you can afford.

This is why models got big. Parallel training is what let anyone spend a hundred times more compute on the same idea.

What a transformer block contains

Attention alone is not enough — three pieces get bolted on.

1

Self-attention

The same mechanism, but each word attends to the other words in its own sentence rather than a separate one.

2

Position signal

Attention has no idea what order words came in. So a position marker is added to each word before it enters.

3

A small network

After attention, each word is processed on its own by a small feed-forward network. Attention mixes; this thinks.